ChatGPT

Arhitectura instrumentelor de management al cazurilor de testare

Rezumat executiv

Instrumentele de tip *test case management* (TCM) gestionează două lumi care trebuie modelate separat: **designul testelor** (cazuri, suite, cerințe) și **execuția testelor** (rulări, rezultate, defecte). În documentația TestRail, distincția este explicită: *cazurile de test* sunt „șabloane", iar *testele* sunt „materializarea" unui caz într-un *test run* (instanțe execuționale), care agregă apoi unul sau mai multe *rezultate* (attempt-uri). [1]

Această separare se regăsește și în instrumente enterprise de ALM: în OpenText [2] Application Quality Management/ALM Quality Center există module distincte pentru **Requirements**, **Test Plan**, **Test Lab**, **Test Runs** și **Defects**, iar legăturile (coverage/traceability) se fac între cerințe, teste și defecte. [3]

În practică, un TCM matur trebuie să ofere: (a) un **model de date** consistent pentru proiecte, suite, planuri, rulări și rezultate; (b) **workflows** și **status-uri** configurabile; (c) **câmpuri custom** și etichete; (d) **integrare** cu issue trackers (de ex. Jira) și CI/CD; (e) **audit, backup/restore și retenție**; (f) **scalare** pentru volume mari de rezultate și atașamente. TestRail documentează explicit webhooks, audit logging, backup configurabil și integrare cu Jira, ceea ce fixează cerințe arhitecturale clare (evenimente, jurnalizare, stocare, integrare). [4]

Model de domeniu și relații între entități

Principii de modelare a domeniului

Un model robust pornește de la două axiome (utile pentru „explicat altui AI"):

1) **Design vs Execuție**:
- **TestCase** definește *ce* și *cum* testăm (precondiții, pași, așteptări). [5]
- **Test** este instanța unui **TestCase** într-un anumit context execuțional (run/plan/config). [1]
- **TestResult** este un *attempt* (încercare) de execuție pentru un **Test** (poate exista istoric cu multiple rezultate). [1]

2) **Planificare ca agregare**:
- **TestPlan** agregă multiple **TestRun** și poate genera automat runs pe baza unor configurații (OS/browser/etc). [6]
- **Milestone/Release** agregă planuri și/sau rulări în jurul unui prag (release, versiune internă). [7]

Mapare terminologică între produse (pentru interoperabilitate)

În ecosisteme diferite, aceeași intenție apare sub alte nume:

- **Test Plan**: TestRail „Plans" (grupează runs și generează runs pe configurații) [8] ; Xray „Test Plan" (listează Tests și Test Executions) [9] ; Azure DevOps „Test Plan" (work item type) [10] ; qTest „Test Plan" (cu Release/Cycle/Suite/Run) [11] .

- **Test Run / Execuție**: TestRail „Run” (container de tests/results) [12] ; Xray „Test Execution” (execuții asociate unui Test Plan) [13] ; Zephyr „Test Cycle / Test Run” (execuție structurată) [14] ; Azure DevOps folosește **test points** generate când incluzi test cases în plan/suite și execuți acele puncte [15] ; qTest definește explicit Test Runs ca instanțe executate ale Test Cases [16] .

Diagrama ER de referință (model canonical)

Diagrama de mai jos este o **arhitectură de date „canonicală”**: include entitățile cerute și relațiile minime pentru a reproduce comportamente documentate (ex. „tests sunt instanțe ale cases în runs”, „plans agregă runs”, „custom fields pe cases/results”, „audit logging”). [17]

```
erDiagram
    TENANT ||--o{ PROJECT : owns

    PROJECT ||--o{ TEST_SUITE : contains
    TEST_SUITE ||--o{ SECTION : organizes
    SECTION ||--o{ TEST_CASE : includes

    TEST_CASE ||--o{ TEST_CASE_VERSION : versions
    TEST_CASE ||--o{ TEST_CASE_TAG : labeled_by
    TAG ||--o{ TEST_CASE_TAG : applied_to

    PROJECT ||--o{ MILESTONE : milestones
    PROJECT ||--o{ TEST_PLAN : plans
    MILESTONE ||--o{ TEST_PLAN : groups

    TEST_PLAN ||--o{ PLAN_ENTRY : entries
    PLAN_ENTRY ||--o{ TEST_RUN : generates

    PROJECT ||--o{ CONFIG_GROUP : defines
    CONFIG_GROUP ||--o{ CONFIG_OPTION : options

    PROJECT ||--o{ TEST_ENVIRONMENT : environments
    TEST_RUN }o--|| TEST_ENVIRONMENT : executes_in
    TEST_RUN }o--o{ CONFIG_OPTION : executes_on

    TEST_RUN ||--o{ TEST : contains
    TEST_CASE ||--o{ TEST : materializes_as
    TEST ||--o{ TEST_RESULT : has_history

    REQUIREMENT ||--o{ REQUIREMENT_LINK : linked
    TEST_CASE ||--o{ REQUIREMENT_LINK : covers

    DEFECT ||--o{ DEFECT_LINK : linked
    TEST_RESULT ||--o{ DEFECT_LINK : references

    ATTACHMENT ||--o{ ATTACHMENT_LINK : linked
    TEST_CASE ||--o{ ATTACHMENT_LINK : has
    TEST_RESULT ||--o{ ATTACHMENT_LINK : has
```

```
PROJECT ||--o{ CUSTOM_FIELD_DEF : customizes
CUSTOM_FIELD_DEF ||--o{ CUSTOM_FIELD_VALUE : values
TEST_CASE ||--o{ CUSTOM_FIELD_VALUE : has_values
TEST_RESULT ||--o{ CUSTOM_FIELD_VALUE : has_values

PROJECT ||--o{ WORKFLOW : workflows
WORKFLOW ||--o{ STATUS : statuses
STATUS ||--o{ STATUS_TRANSITION : transitions
STATUS ||--o{ STATUS_TRANSITION : transitions_to

USER ||--o{ PROJECT_MEMBERSHIP : member_of
PROJECT ||--o{ PROJECT_MEMBERSHIP : has_members
ROLE ||--o{ PROJECT_MEMBERSHIP : grants_role

PROJECT ||--o{ AUDIT_EVENT : records
USER ||--o{ AUDIT_EVENT : actor
```

Tabel ER detaliat (entități, chei, relații, invariantes)

Tabelul este scris intenționat „rigid”, ca un contract semantic pentru un AI (ce înseamnă fiecare entitate și cum se leagă). În special, relația **TestCase → Test → TestResult** urmează exact descrierea din TestRail: run creează „test” pentru fiecare case, iar rezultatele se atașează la test/case+run. [18]

Entitate	Cheie primară (ex.)	Atribute critice	Relații principale	Invariante utile pentru AI
Tenant/ Organization	tenant_id	nume, plan/ licență, politici retenție	1:N Projects	Toată izolarea multi-tenant se ancorează pe tenant_id. [19]
Project	project_id	nume, tip proiect, setări (statusuri, câmpuri)	N:1 Tenant; 1:N Suites/ Plans/Runs/ Cfg/Users	În TestRail „Project” este unitatea organizațională de top. [20]
TestSuite	suite_id	nume, mod repo (single/multi)	N:1 Project; 1:N Sections/ Cases	În proiecte multi-suite, suite-urile sunt izolate („no relationship whatsoever”). [21]
Section	section_id	parent_section_id, path	N:1 Suite; 1:N Cases	Section este ierarhic (secțiuni/subsecțiuni). [21]
TestCase	case_id	titlu, precondiții, pași, template, prioritate, referințe	N:1 Section; 1:N Versions; M:N Requirements; M:N Tags; M:N Attachments	„Case fields și templates pot fi complet customizate” (TestRail). [22]

Entitate	Cheie primară (ex.)	Atribute critice	Relații principale	Invariante utile pentru AI
TestCaseVersion	case_version_id	snapshot fields, diff refs, created_by	N:1 TestCase	Versioning & restore sunt funcții dedicate în TestRail Enterprise. [23]
Milestone/ Release	milestone_id	start/end, descriere	N:1 Project; 1:N Plans/ Runs	Milestone grupează activități și crește trasabilitatea. [24]
TestPlan	plan_id	nume, milestone_id, descriere	N:1 Project; 1:N PlanEntry; 1:N Runs (via entries)	Plan grupează runs și generează runs pe configurații. [25]
PlanEntry	entry_id	suite_id, includere cazuri (all/ selection), cfg matrix	N:1 Plan; 1:N Runs	În TestRail, plan entry este container pentru runs + configurații. [26]
TestRun	run_id	state (open/ closed/archived), assigned_to, selection	N:1 Project; N:1 PlanEntry (opțional); 1:N Tests; M:N ConfigOptions; N:1 Environment	Închidere run arhivează tests & results; smart archiving introduce „Archived status”. [27]
Test	test_id	case_id, run_id, status curent, assigned_to	N:1 Run; N:1 Case; 1:N Results	„Tests sunt instanțe ale cases în runs; tests agregă results.” [28]
TestResult	result_id	status_id, comment, defects refs, created_by, created_on	N:1 Test; M:N Defects; M:N Attachments	Status-urile standard includ Passed/ Blocked/Untested/ Retest/Failed și au ID. [29]
Requirement	req_id	tip (internal/ external), external_key	M:N Cases; M:N Defects	ALM: cerințele pot fi legate de teste și defecte; coverage evidențiază impact. [30]
Defect	defect_id	external_key (Jira), status, assignee	M:N Results; M:N Requirements (opțional)	TestRail: integrarea cu Jira permite link ca defect/reference și creare issue din TestRail. [31]

Entitate	Cheie primară (ex.)	Atribute critice	Relații principale	Invariante utile pentru AI
TestEnvironment	env_id	nume, endpoints, variabile, sensibilitate	1:N Runs	În practică e distinct de Config: env = „unde”, config = „pe ce combinație”. (Model de referință; vezi configurări multiple în TestRail/Azure/ALM.) [32]
ConfigGroup/ Option	config_id	ex. Browser/OS; valori	M:N Runs/ Plans	TestRail: configurații pe grupuri/opțiuni, folosite la generare runs. [33]
Attachment	attachment_id	storage_key, mime, size, checksum	M:N Entities	TestRail API: upload multipart/form-data pentru attachments. [34]
User/Role/ Permission	user_id / role_id	auth, identitate, membership	M:N Projects via membership	TestGrid documentează RBAC pe roluri (Company Admin, Cloud Admin etc.). [35]
CustomField (def+value)	field_id / value_id	scope (case/ result), type	N:1 Project; N: 1 Entity	TestRail suportă custom fields pentru *cases* și *results*. [36]
Workflow/ Statuses	workflow_id	transitions, rules	N:1 Project; 1:N Statuses	TestRail: statusurile pentru test case approvals (Design/ Review/Ready) sunt configurabile. [37]
AuditEvent	audit_id	actor, entity, before/after, timestamp	N:1 Project; N: 1 User	TestRail: audit logging poate înregistra create/update/delete pentru entități. [38]

Fluxuri operaționale cheie

Creare test plan și generare test runs

În TestRail, un plan grupează rulări și poate genera automat rulări pentru configurații (browser/OS/alte configurații). [39]

```
sequenceDiagram
  autonumber
```

```
  actor TL as Test Lead
  participant UI as Web UI
  participant API as API Gateway
  participant AUTH as AuthN/Z
  participant PLAN as Planning Service
  participant CASE as Case Repo Service
  participant EXEC as Execution Service
  participant DB as Relational DB
  participant IDX as Search/Index
  participant EVT as Webhook/Event Dispatcher

  TL->>UI: Definește Plan (milestone, suite, selecție cazuri, config matrix)
  UI->>API: POST /plans + entries + configs
  API->>AUTH: Verifică drepturi (create plan/run)
  AUTH-->>API: OK
  API->>PLAN: createPlan(plan, entries)
  PLAN->>DB: INSERT plan, entries
  PLAN->>CASE: resolveCaseSelection(suite, filters/manual ids)
  CASE->>DB: SELECT cases
  CASE-->>PLAN: case_ids
  PLAN->>EXEC: createRunsFromEntries(case_ids, configs)
  EXEC->>DB: INSERT runs
  EXEC->>DB: INSERT tests (instanțe ale cazurilor în run)
  EXEC-->>PLAN: run_ids
  PLAN->>IDX: index(plan, runs, cases refs)
  PLAN->>EVT: emit events (PLAN_CREATED, RUN_CREATED)
  API-->>UI: 201 Created (plan_id, runs)
```

Punctul arhitectural critic: odată cu crearea run-ului, se creează și **Test**-urile ca instanțe ale cazurilor (TestCase → Test). Acest comportament este descris explicit în TestRail (run creează un test pentru fiecare case din suite). [18]

Execuție test run și înregistrare rezultate

Execuția are două sub-modele:

- **Manual**: utilizatorul actualizează statusul și adaugă un rezultat; în ALM acest lucru este în zona „Test Lab” (run tests), iar rezultatele sunt apoi analizate. [40]
- **Automat**: un CI/test runner trimite rezultate prin API; TestRail subliniază că API-ul e folosit frecvent pentru a trimite rezultate din teste automate. [41]

```
sequenceDiagram
  autonumber
  actor QA as Tester/CI
  participant UI as Web UI / CI Client
  participant API as API Gateway
  participant AUTH as AuthN/Z
  participant EXEC as Execution Service
  participant DB as Relational DB
  participant IDX as Search/Index
```

```
  participant EVT as Webhook/Event Dispatcher
  participant RPT as Reporting/Analytics

  QA->>UI: Deschide Run / pornește execuția
  UI->>API: GET /runs/{run_id}/tests
  API->>AUTH: rights(check read/execute)
  AUTH-->>API: OK
  API->>DB: SELECT tests + ultimele rezultate
  API-->>UI: Lista de Tests (instance-of-case)

  QA->>UI: Înregistrează rezultat (status, comment, defects, attachments)
  UI->>API: POST /runs/{run_id}/results (bulk dacă e CI)
  API->>AUTH: rights(add_results)
  AUTH-->>API: OK
  API->>EXEC: addResults(run_id, results[])
  EXEC->>DB: INSERT test_results (istoric)
  EXEC->>DB: UPDATE tests.current_status
  EXEC->>RPT: recompute progress (pass rate, remaining)
  EXEC->>IDX: index new result text/metadata
  EXEC->>EVT: emit RESULT_CREATED / TEST_UPDATED
  API-->>UI: 200 OK
```

Note performanță (documentate): în TestRail Cloud există limitări de tip rate limit și e recomandat să folosești endpoint-uri bulk pentru a evita rate limits (ex. add_results_for_cases). [42]

Raportare defecte și trasabilitate către issue tracker

Un TCM modern trebuie să suporte:
- **linking** defecte existente (ID extern) la rezultate/cazuri;
- **crearea** unui defect în issue tracker direct din TCM;
- păstrarea trasabilității între requirement ↔ test ↔ defect.

TestRail oferă integrare cu Atlassian [43] Jira pentru a lega issues ca *requirements* sau *defects* și pentru a crea noi issues din TestRail, plus acoperire (coverage/traceability). [44]
În ALM, defectele pot fi legate de requirements/tests/runs/run steps etc. [45]
În qTest, există flux explicit de submit defect din contextul execuției (Test Log / Test Step). [46]

```
sequenceDiagram
  autonumber
  actor QA as Tester
  participant UI as Web UI
  participant API as API Gateway
  participant AUTH as AuthN/Z
  participant DEF as Defect/Integration Service
  participant DB as Relational DB
  participant JIRA as Issue Tracker (Jira)
  participant EVT as Webhook/Event Dispatcher

  QA->>UI: "Create Defect" dintr-un rezultat eșuat
  UI->>API: POST /defects (or /results/{id}/defects)
```

```
  API->>AUTH: rights(create_defect)
  AUTH-->>API: OK

  API->>DEF: createOrLinkDefect(result_id, payload)
  alt Creare defect extern
    DEF->>JIRA: Create Issue (summary, steps, env, attachments refs)
    JIRA-->>DEF: issue_key, status
  else Link defect existent
    DEF->>JIRA: GET Issue(issue_key) (optional verify)
    JIRA-->>DEF: issue status/fields
  end
  DEF->>DB: INSERT defect + link (result_id <-> defect_key)
  DEF->>EVT: emit DEFECT_LINKED / DEFECT_CREATED
  API-->>UI: 200 OK (defect_key)
```

Mecanismul de integrare poate fi implementat ca „plugin”/conector; TestRail descrie defect plugins care pot „push” bug reports folosind API-ul tracker-ului și pot face lookup din TestRail. [47]

Arhitectură de componente și integrare

Componente logice (container/component view)

O arhitectură de referință (compatibilă cu features documentate în TestRail/ALM/TestGrid/Azure) conține următoarele „blocuri”:

- **Frontend Web**: ecran proiect/suite/case/run/rapoarte; multe sisteme sunt web-based (TestRail este explicit web-based). [48]
- **API Layer** (REST, uneori GraphQL): pentru UI, CI/CD și integrări; TestRail menționează API HTTP-based cu JSON/UTF-8. [49]
- **Servicii de domeniu**:
- Case Repository (suite/sections/cases/versioning/custom fields) [50]
- Planning (milestones/plans/configs) [51]
- Execution (runs/tests/results/statuses/archiving) [52]
- Defects/Requirements Integration (Jira, etc.) [53]
- **Relational DB**: referință pentru entități tranzacționale (integritate + interogări).
- **Search/Index**: full-text și relevanță; TestRail descrie că poți căuta orice entitate și oferă predicții/relevance scores. [54]
- **Object Storage**: atașamente (screenshots, logs); TestRail are API pentru attachments și upload multipart. [34]
- **Eventing/Webhooks**: notificări real-time către alte sisteme; TestRail are funcție de webhooks la nivel global sau pe proiect. [55]
- **CI/CD Hooks**: import rezultate; TestGrid explică integrarea prin apel API din CI (ex. Jenkins) într-un flux în 3 pași, iar test results includ logs/screenshots/build summary. [56]

```
flowchart LR
  subgraph Client
    UI[Web Frontend]
    CI[CI/CD + Test Runners]
  end
```

```
subgraph Edge
  GW[API Gateway / BFF]
  AUTH[Auth Service (SSO/OIDC/SAML)]
end

subgraph CoreDomain
  CASE[Case Repository Service]
  PLAN[Planning Service]
  EXEC[Execution Service]
  INTEG[Integrations Service\n(Issue trackers, Requirements)]
  RPT[Reporting & Analytics Service]
  ADM[Admin/Customization Service\n(Custom fields, statuses, workflows)]
end

subgraph Data
  DB[(Relational DB)]
  IDX[(Search Index)]
  OBJ[(Object Storage)]
  LOGS[(Central Logs/Audit Store)]
end

subgraph Events
  BUS[(Event Bus / Queue)]
  WH[Webhook Dispatcher]
end

UI --> GW
CI --> GW
GW --> AUTH
GW --> CASE
GW --> PLAN
GW --> EXEC
GW --> INTEG
GW --> RPT
GW --> ADM

CASE --> DB
PLAN --> DB
EXEC --> DB
ADM --> DB
INTEG --> DB
RPT --> DB

CASE --> IDX
EXEC --> IDX
RPT --> IDX

EXEC --> OBJ
CASE --> OBJ

EXEC --> BUS
```

```
PLAN --> BUS
INTEG --> BUS
BUS --> WH
WH --> External[External Systems\n(Jira, Slack, BI, etc.)]

GW --> LOGS
EXEC --> LOGS
ADM --> LOGS
```

Diagrama urmează modelul „containers/components” popularizat de **C4 model** (set ierarhic de diagrame pentru comunicarea arhitecturii). [57]

Exemple de contract API (REST și GraphQL)

Aceste exemple sunt **propuneri de contract** (nu „copie” 1:1 dintr-un produs), dar sunt compatibile cu semantica documentată: plan→runs (cu configs), run→tests, tests→results, attachments multipart, bulk results, webhooks. [58]

REST (exemplu)

```
POST /api/v1/projects
Content-Type: application/json

{
  "tenantId": "t-123",
  "name": "Payments Platform",
  "key": "PAY",
  "defaultWorkflowId": "wf-default"
}
```

```
POST /api/v1/projects/PAY/plans
Content-Type: application/json

{
  "name": "Regression - Release 2026.04",
  "milestoneId": "ms-2026-04",
  "entries": [
    {
      "suiteId": "suite-core",
      "includeAllCases": false,
      "caseIds": [101, 102, 205],
      "configurationOptionIds": ["cfg.browser.chrome", "cfg.browser.firefox"]
    }
  ]
}
```

```
POST /api/v1/runs/run-778/results:bulk
Content-Type: application/json
```

```
{
  "source": "CI",
  "results": [
    {
      "caseId": 101,
      "status": "PASSED",
      "startedAt": "2026-03-27T08:10:00Z",
      "finishedAt": "2026-03-27T08:10:12Z",
      "comment": "JUnit imported",
      "defects": ["JIRA-1234"],
      "customFields": {
        "cf_build": "2026.04.0+145",
        "cf_commit": "a1b2c3d"
      }
    }
  ]
}
```

```
POST /api/v1/results/res-990/attachments
Content-Type: multipart/form-data; boundary=----XYZ

------XYZ
Content-Disposition: form-data; name="file"; filename="screenshot.png"
Content-Type: image/png

(binary)
------XYZ--
```

GraphQL (exemplu)

```
type Query {
  project(key: ID!): Project
  testRun(id: ID!): TestRun
}

type Project {
  key: ID!
  suites: [TestSuite!]!
  milestones: [Milestone!]!
}

type TestRun {
  id: ID!
  state: RunState!
  tests(status: TestStatus, assigneeId: ID): [Test!]!
  progress: RunProgress!
}
```

```
type Mutation {
  createPlan(input: CreatePlanInput!): TestPlan!
  addResult(input: AddResultInput!): TestResult!
}
```

Integrare: CI/CD, webhooks, sisteme externe

- **Webhooks**: într-un model event-driven, orice mutație relevantă (CASE_CREATED, RUN_CREATED, RESULT_CREATED) produce un eveniment; TestRail documentează configurarea webhooks global sau per proiect. [55]
- **CI/CD**: TestGrid descrie integrarea prin apelarea unui CI API dintr-un tool precum Jenkins și folosește tokeni/parametri specifici; aceasta implică un endpoint stabil și autentificare de tip token. [56]
- **Issue trackers**: integrarea cu Jira în TestRail include link-uri bidirecționale și posibilitatea de „create new Jira issues from within TestRail". [59]
- **Requirements**: pentru coverage, ALM descrie legarea tests↔requirements; Azure DevOps folosește „Related Work" și stochează obiectele de test ca work items, ceea ce cere un model de referință clar pentru link-uri. [60]

Implementare și operare: deployment, scalare, securitate, audit, backup și performanță

Opțiuni de deployment

- **SaaS multi-tenant**: necesită o strategie de izolare între tenanți (pool/silo/bridge). Documentația Amazon Web Services [61] (SaaS tenant isolation) descrie modele tip pool vs silo și trade-off-uri, iar Microsoft [62] (Azure SQL) discută explicit modele de tenancy și impactul asupra designului și operațiunilor. [63]
- **On-prem**: multe organizații aleg control complet asupra DB/attachments; TestRail documentează diferențe Server vs Cloud și include considerente de autentificare și backup. [64]
- **Hybrid**: de obicei *control plane* (SaaS) + *data plane* (on-prem pentru atașamente/loguri) sau conectori; justificarea vine din cerințe de rezidență date, performanță locală, sau integrare în rețele restricționate (model de referință; vezi și practicile de izolare și tenancy). [65]

Scalabilitate și performanță

Problema majoră de scalare în TCM este volumul de **runs/tests/results** și impactul asupra UI/rapoarte:

- TestRail introduce „Smart Archiving" pentru a reduce „clutter" și impactul asupra performanței pe volume mari de runs/plans istorice; arhivarea are reguli (auto close după un interval, auto archive după alt interval) și status „Archived" (locked, excluded from reports, still searchable). [66]
- La nivel API, există rate limits și cod 429 în cloud; documentația recomandă endpoint-uri bulk și/sau delay pentru a evita limitările. [67]
- Search trebuie separat de DB tranzacțională pentru experiență bună: TestRail menționează că search poate căuta orice entitate și folosește predicții cu relevance scores, ceea ce sugerează fie index dedicat, fie mecanisme avansate peste DB. [54]

Securitate: identitate, autorizare, protecția datelor

Un TCM gestionează date sensibile (cerințe, rezultate, defecte, loguri, uneori evidențe de conformitate). Arhitectura trebuie să includă:

- **Autorizare centralizată** (Policy Enforcement Point) și principiul „deny by default”; OWASP [68] recomandă să forțezi toate cererile printr-un strat de verificare a accesului și să consolidezi logica de access control. [69]
- **RBAC/ABAC**: OWASP descrie RBAC și ABAC ca forme larg utilizate; TestGrid oferă un exemplu concret de RBAC pe roluri (Company Admin/Cloud Admin/Team Admin/Team User) și separă responsabilități. [70]
- **SSO**: TestRail are funcție de SSO (Enterprise) [71] ; OpenText ALM/QC menționează suport SAML/OpenID pentru SSO. [72]
- **Token-uri**: dacă folosești JWT pentru API, standardul e definit în RFC 7519 (claims, semnare/encrypt). [73]

Audit și logging

- TestRail documentează audit logging (Enterprise) care poate înregistra create/update/delete pentru entități, pentru trail de activitate (compliance, accountability, troubleshooting). [38]
- Pentru practici generale, NIST SP 800-92 tratează log management ca disciplină (colectare, stocare, analiză, retenție), iar OWASP subliniază importanța logging-ului pentru evenimente de securitate. [74]

Backup/restore, retenție și migrare

- TestRail: backup-ul trebuie să includă DB + fișiere/instalare + attachments (Server) [75] ; există și ghid de restore (inclusiv migrare cloud→local) [76] ; în cloud există backup zilnic (Professional) și opțiuni configurabile + restore on-demand (Enterprise). [77]
- TestRail Cloud are și o pagină de „Data Storage and Management” (storage, exports, attachments), inclusiv observații despre actualizarea calculului de storage și efecte după ștergere (până la 24h). [78]
- Pentru migrare între platforme, există ghid oficial de „Migrate from TestRail Server to TestRail Cloud”. [79]
- OpenText ALM/QC: FAQ-ul recomandă upgrade într-un mediu staging pentru a estima durata și a minimiza riscul; durata depinde de factori precum numărul de proiecte, utilizatori, dimensiunea proiectelor și atașamente. [80]

Strategie de retenție (model de referință inspirat din practici documentate):
- *Hot data*: runs active + ultimele rezultate (folosite la rapoarte).
- *Warm data*: runs închise, încă relevante pentru release.
- *Archived*: locked, exclus din rapoarte dar păstrat pentru referință și search (exact cum descrie Smart Archiving). [81]
- *Purged*: șters definitiv, cu politici per tenant și audit al acțiunii (în practică, trebuie corelat cu cerințe legale/compliance; vezi importanța log management). [82]

Bune practici recomandate și comparație între instrumente

Comparație (feature + implicație arhitecturală)

Tabelul de mai jos compară **alegeri arhitecturale** observabile în documentații oficiale sau primare. (Nu e un „review” de produs; e o mapare din perspective de arhitectură.) [83]

Familie / exemplu	Entități execuție	Requirements & coverage	Workflow/ status	Versioning & istoric	Integrare & eventing	Deploy operar
TestRail (Gurock)	Run → Tests (instanțe) → Results (istoric) [1]	„References” + integrare defect/ requirements (ex. Jira) [44]	Status IDs pentru rezultate; approvals pentru test cases [84]	Test case versioning + restore [23]	Webhooks + defect plugins + API HTTP/JSON [85]	Backup (cloud/ server) smart archivi data m [86]
OpenText ALM/QC	Test Lab/ Test Sets/ Run Tests + Test Runs module [40]	Requirements ↔ tests ↔ defects; test coverage [87]	Workflow scripts (JS) + Project Customization [88]	(Istoric prin entități + audit la nivel platformă; depinde de configurație)	REST API pe entități (defect, requirement, test, run) [89]	On-pre SaaS mențio upgrad instala ghidat
TestGrid	Execuție + rezultate cu logs/ screenshots/ build summary [91]	Integrare Jira (doc dedicat) [92]	RBAC pe roluri (4 roluri) [35]	„Test Case Versioning & Management” (concept + versiuni) [93]	CI/CD integration prin CI API + exemplu Jenkins [94]	(Mode manag useri/e dispoz [35]
Xray (Jira app)	Test Plan include Tests și Test Executions [9]	Bazat pe Jira work item types; suport plan/design/ execuție [95]	Urmează workflow Jira (statusuri issues)	(Depinde de Jira issue history)	Integrare nativă Jira (fiind app)	Jira Clo (în fun instala
Azure Test Plans	Test Plans/ Suites/Cases ca work items; test points generate [96]	Link „Related Work” la cerințe/ bugs; trasabilitate Git [10]	Bazat pe permisiuni Azure DevOps + proces customization [10]	(Istoric work items)	Integrare cu pipelines; configurări de test [97]	SaaS (A DevOp on-pre (Azure DevOp Server
qTest (Tricentis)	Test Runs = instanțe executate ale Test Cases [16]	Link requirements↔test cases; planificare pe cycles/suites [11]	(Workflows interne modul defects) [98]	History tracking pentru requirements/test cases/suites/runs/ defects [99]	Submit defect din execuție + integrări CI [46]	SaaS/o prem î funcție ediție

Best practices (arhitectură + modelare, orientat AI)

1) **Păstrează explicit triada Case–Test–Result**. În modelul TestRail, aceasta evită confuzii: aceeași cerință/caz poate fi executată în mai multe rulări (deci mai multe „tests”), iar fiecare test poate avea istoric de rezultate. [1]

2) **Planificarea trebuie să suporte matrice de configurații** (OS/browser/device). TestRail generează runs din configs, iar Azure Test Plans explică asignarea de configurații și urmărirea rezultatelor per combinație. [100]

3) **Câmpuri custom și extensibilitate fără migrații frecvente de DB**. TestRail suportă custom fields pentru cases și results; arhitectural, asta cere fie JSONB, fie tabele EAV, fie „typed custom field tables" (alegere de implementare). [36]

4) **Workflow configurabil pentru calitatea repository-ului**. TestRail are explicit workflow de review & approvals pentru test cases (statusuri și aprobări per proiect). [37]

5) **Versioning și audit ca funcții de produs, nu „addon"**. TestRail oferă compare/restore la versiuni de test case; qTest indică history tracking pentru obiecte cheie. [101]

6) **Eventing standardizat**: webhooks + queue intern. Webhooks (TestRail) cer un subsistem de livrare robust (retry, semnare, rate limit). [55]

7) **Retenție și performanță prin lifecycle management**. Smart archiving (TestRail) arată o abordare practică: automatizare close/archive, status „Archived", excludere din rapoarte, dar păstrare pentru search și rerun. [66]

8) **Securitate**: access control centralizat + RBAC/ABAC + audit logs. OWASP recomandă consolidarea verificărilor de acces și logging pentru evenimente; NIST oferă ghid pentru log management enterprise. [102]

Surse prioritare (linkuri)

```
https://support.testrail.com/hc/en-us/articles/15758177606676-API-uses-cases-
intro
https://support.testrail.com/hc/en-us/articles/7077711537684-Plans
https://support.testrail.com/hc/en-us/articles/7077874763156-Runs
https://support.testrail.com/hc/en-us/articles/7077819312404-Results
https://support.testrail.com/hc/en-us/articles/7373850291220-Configuring-
custom-fields
https://support.testrail.com/hc/en-us/articles/7619613118484-Audit-logging
https://support.testrail.com/hc/en-us/articles/7169038183572-Webhooks
https://support.testrail.com/hc/en-us/articles/46348615843476-Smart-
Archiving-for-Test-Runs-Plans
https://admhelp.microfocus.com/alm/en/25.1/pdfs/Tutorial.pdf
https://admhelp.microfocus.com/alm/api_refs/REST_core/Content/REST_API_Core/
General/Overview.html
https://docs.getxray.app/space/XRAYCLOUD/44565153/Test%2BPlan
https://learn.microsoft.com/en-us/azure/devops/test/test-objects-overview?
view=azure-devops
https://docs.tricentis.com/qtest-saas/content/manager/introduction/
introduction_to_qtest_objects.htm
https://support.smartbear.com/zephyr-scale-cloud/api-docs/
```

---

1 5 12 17 21 22 26 28 50 68 83 https://support.testrail.com/hc/en-us/articles/15758177606676-API-uses-cases-intro
https://support.testrail.com/hc/en-us/articles/15758177606676-API-uses-cases-intro

2 15 https://learn.microsoft.com/en-us/azure/devops/test/navigate-test-plans?view=azure-devops
https://learn.microsoft.com/en-us/azure/devops/test/navigate-test-plans?view=azure-devops

3 https://admhelp.microfocus.com/alm/en/25.1/pdfs/Tutorial.pdf
https://admhelp.microfocus.com/alm/en/25.1/pdfs/Tutorial.pdf

4 55 85 Webhooks
https://support.testrail.com/hc/en-us/articles/7169038183572-Webhooks?utm_source=chatgpt.com

6 8 25 39 51 100 https://support.testrail.com/hc/en-us/articles/7077711537684-Plans
https://support.testrail.com/hc/en-us/articles/7077711537684-Plans

7 24 Milestones
https://support.testrail.com/hc/en-us/articles/15545364561044-Milestones?utm_source=chatgpt.com

9 https://getxraydocs.atlassian.net/wiki/spaces/XRAYCLOUD/pages/44565153/Test%2BPlan
https://getxraydocs.atlassian.net/wiki/spaces/XRAYCLOUD/pages/44565153/Test%2BPlan

10 96 https://learn.microsoft.com/en-us/azure/devops/test/test-objects-overview?view=azure-devops
https://learn.microsoft.com/en-us/azure/devops/test/test-objects-overview?view=azure-devops

11 https://docs.tricentis.com/qtest-saas/content/manager/introduction/qtest_manager_quick_start_guides.htm
https://docs.tricentis.com/qtest-saas/content/manager/introduction/qtest_manager_quick_start_guides.htm

13 https://docs.getxray.app/space/XRAYCLOUD/44565117
https://docs.getxray.app/space/XRAYCLOUD/44565117

14 https://zephyrdocs.atlassian.net/wiki/spaces/ZFJCLOUD/pages/21757957/Planning%2BTest%2BCycles
https://zephyrdocs.atlassian.net/wiki/spaces/ZFJCLOUD/pages/21757957/Planning%2BTest%2BCycles

16 62 https://docs.tricentis.com/qtest-saas/content/manager/introduction/introduction_to_qtest_objects.htm
https://docs.tricentis.com/qtest-saas/content/manager/introduction/introduction_to_qtest_objects.htm

18 https://support.testrail.com/hc/en-us/articles/7077819312404-Results
https://support.testrail.com/hc/en-us/articles/7077819312404-Results

19 63 https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/pool-isolation.html
https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/pool-isolation.html

20 Projects and their types
https://support.testrail.com/hc/en-us/articles/7076923860244-Projects-and-their-types?utm_source=chatgpt.com

23 101 Test case versioning
https://support.testrail.com/hc/en-us/articles/7768433966996-Test-case-versioning?utm_source=chatgpt.com

27 52 67 https://support.testrail.com/hc/en-us/articles/7077874763156-Runs
https://support.testrail.com/hc/en-us/articles/7077874763156-Runs

29 84 Statuses
https://support.testrail.com/hc/en-us/articles/7077935129364-Statuses?utm_source=chatgpt.com

30 https://admhelp.microfocus.com/alm/en/latest/online_help/Content/UG/t_use_requirements.htm
https://admhelp.microfocus.com/alm/en/latest/online_help/Content/UG/t_use_requirements.htm

31 43 44 53 59 Integrate with Jira
https://support.testrail.com/hc/en-us/articles/7664835909524-Integrate-with-Jira?utm_source=chatgpt.com

32 33 Configurations
https://support.testrail.com/hc/en-us/articles/23043143013268-Configurations?utm_source=chatgpt.com

34 Attachments
https://support.testrail.com/hc/en-us/articles/7077196481428-Attachments?utm_source=chatgpt.com

35 https://testgrid.io/docs/document/testgrid-user-role-management/
https://testgrid.io/docs/document/testgrid-user-role-management/

36 Configuring custom fields
https://support.testrail.com/hc/en-us/articles/7373850291220-Configuring-custom-fields?utm_source=chatgpt.com

37 Test case review & approvals
https://support.testrail.com/hc/en-us/articles/7766980011028-Test-case-review-approvals?utm_source=chatgpt.com

38 Audit logging
https://support.testrail.com/hc/en-us/articles/7619613118484-Audit-logging?utm_source=chatgpt.com

40 Test Lab - ADM Help Centers
https://admhelp.microfocus.com/alm/en/latest/online_help/Content/UG/t_run_tests.htm?utm_source=chatgpt.com

41 49 58 https://support.testrail.com/hc/en-us/articles/7077083596436-Introduction-to-the-TestRail-API
https://support.testrail.com/hc/en-us/articles/7077083596436-Introduction-to-the-TestRail-API

42 https://support.testrail.com/hc/en-us/articles/7077175066388-Error-handling
https://support.testrail.com/hc/en-us/articles/7077175066388-Error-handling

45 https://genmab.tmls.saas.nnit.com/qcbin/Help/doc_library/online_help/Content/UG/t_track_defects.htm
https://genmab.tmls.saas.nnit.com/qcbin/Help/doc_library/online_help/Content/UG/t_track_defects.htm

46 https://docs.tricentis.com/qtest-saas/content/manager/defects/submit_defects_in_test_execution.htm
https://docs.tricentis.com/qtest-saas/content/manager/defects/submit_defects_in_test_execution.htm

47 Configuring defect integrations
https://support.testrail.com/hc/en-us/articles/7747085183636-Configuring-defect-integrations?utm_source=chatgpt.com

48 https://support.testrail.com/hc/en-us/articles/7076810203028-Introduction-to-TestRail
https://support.testrail.com/hc/en-us/articles/7076810203028-Introduction-to-TestRail

54 https://support.testrail.com/hc/en-us/articles/7827954050836-Search
https://support.testrail.com/hc/en-us/articles/7827954050836-Search

56 94 https://testgrid.io/docs/document/ci-cd-integration/
https://testgrid.io/docs/document/ci-cd-integration/

57 https://c4model.com/
https://c4model.com/

60 87 https://admhelp.microfocus.com/alm/en/25.1/online_help/Content/UG/t_create_test_coverage.htm
https://admhelp.microfocus.com/alm/en/25.1/online_help/Content/UG/t_create_test_coverage.htm

61 66 81 https://support.testrail.com/hc/en-us/articles/46348615843476-Smart-Archiving-for-Test-Runs-Plans
https://support.testrail.com/hc/en-us/articles/46348615843476-Smart-Archiving-for-Test-Runs-Plans

64 TestRail Server vs TestRail Cloud
https://support.testrail.com/hc/en-us/articles/16548874084628-TestRail-Server-vs-TestRail-Cloud?utm_source=chatgpt.com

65 https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/silo-pool-and-bridge-models.html
https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/silo-pool-and-bridge-models.html

69 102 https://top10proactive.owasp.org/the-top-10/c1-accesscontrol/
https://top10proactive.owasp.org/the-top-10/c1-accesscontrol/

70 https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

71 Configure SSO
https://support.testrail.com/hc/en-us/articles/7649434769812-Configure-SSO?utm_source=chatgpt.com

72 88 90 https://www.opentext.com/assets/documents/en-US/pdf/alm-quality-center-whats-new-en.pdf
https://www.opentext.com/assets/documents/en-US/pdf/alm-quality-center-whats-new-en.pdf

73 https://datatracker.ietf.org/doc/html/rfc7519
https://datatracker.ietf.org/doc/html/rfc7519

74 82 https://csrc.nist.gov/pubs/sp/800/92/final
https://csrc.nist.gov/pubs/sp/800/92/final

75 Creating a backup of your TestRail Server instance
https://support.testrail.com/hc/en-us/articles/7146305166356-Creating-a-backup-of-your-TestRail-Server-instance?utm_source=chatgpt.com

76 Restoring a TestRail backup
https://support.testrail.com/hc/en-us/articles/7146355895828-Restoring-a-TestRail-backup?utm_source=chatgpt.com

77 86 Hosted TestRail Professional Daily Backups
https://support.testrail.com/hc/en-us/articles/15948314591252-Hosted-TestRail-Professional-Daily-Backups?utm_source=chatgpt.com

78 Data Storage and Management
https://support.testrail.com/hc/en-us/articles/7614836775956-Data-Storage-and-Management?utm_source=chatgpt.com

79 Migrate from TestRail Server to TestRail Cloud
https://support.testrail.com/hc/en-us/articles/31664877921044-Migrate-from-TestRail-Server-to-TestRail-Cloud?utm_source=chatgpt.com

80 https://www.opentext.com/en/media/faq/alm-quality-center-installation-and-upgrade-faq-en.pdf
https://www.opentext.com/en/media/faq/alm-quality-center-installation-and-upgrade-faq-en.pdf

89 https://admhelp.microfocus.com/alm/api_refs/REST_core/Content/REST_API_Core/General/Overview.html
https://admhelp.microfocus.com/alm/api_refs/REST_core/Content/REST_API_Core/General/Overview.html

91 https://testgrid.io/docs/mobile-test-results/
https://testgrid.io/docs/mobile-test-results/

92 https://testgrid.io/docs/document/integration-with-jira/
https://testgrid.io/docs/document/integration-with-jira/

93 https://testgrid.io/docs/test-case-versioning-management/
https://testgrid.io/docs/test-case-versioning-management/

95 https://www.atlassian.com/devops/testing-tutorials/jira-xray-integration-manage-test-cases
https://www.atlassian.com/devops/testing-tutorials/jira-xray-integration-manage-test-cases

97 https://learn.microsoft.com/en-us/azure/devops/test/test-different-configurations?view=azure-devops
https://learn.microsoft.com/en-us/azure/devops/test/test-different-configurations?view=azure-devops

98 https://docs.tricentis.com/qtest-saas/content/manager/defects/using_the_defects_module.htm
https://docs.tricentis.com/qtest-saas/content/manager/defects/using_the_defects_module.htm

99 https://docs.tricentis.com/qtest-saas/content/manager/introduction/qtest_manager_101.htm
https://docs.tricentis.com/qtest-saas/content/manager/introduction/qtest_manager_101.htm